b) Melyek a Observer tervezési minta megvalósításának kellékei? Jelölje meg a helyes válaszokat! Rossz válasz megadása pontlevonással jár. (3p)

- ☐ Alapértelmezett konstruktőr
- ☑ Szubszkripciós (feliratkozási) metódus
- ☑ Unsubscribe (leiratkozási) metódus
- ☑ Változások közvetítése események segítségével
- ☐ Eseménykezelő osztály
- ☑ Különböző típusú Observer-ek
- ☐ Különböző típusú Subject-ek
- ☐ Publikus konstruktor

A fenti válaszokhoz hasonló kérdést kaphatsz a minták működéséről. Ha szeretnél egy másik mintából kérdést, nyugodtan jelezd, és biztosan segítek!

1. Melyek a CommandProcessor tervezési minta fő előnyei? Jelölje meg a helyes válaszokat! Rossz válasz megadása pontlevonással jár. (3p)

- ☑ A parancsok végrehajtásának központosítása
- ☑ Az alkalmazás logikájának elkülönítése a parancs végrehajtásától
- ☑ A kód ismétlésének minimalizálása
- ☐ Az üzleti logika összetettségének növelése
- ☐ A hibák könnyebb kezelése

4. Melyek a CommandProcessor minta legfontosabb jellemzői? Jelölje meg a helyes válaszokat! Rossz válasz megadása pontlevonással jár. (3p)

- ☑ Különválasztja a parancsok végrehajtásának logikáját a hívó kódtól
- ☑ Lehetővé teszi parancsok sorba állítását és késleltetett végrehajtását
- ☐ Minden parancsot egyedileg végrehajt, függetlenül más parancsoktól
- ☐ A parancsok egyetlen végrehajtása után azonnali kimenetet biztosít
- ☑ Az UndoCommand() metódus lehetőséget biztosít a parancsok visszavonására

Kérdés 1: Melyek a Separation of Concerns (SoC) elvének fő előnyei? Jelölje meg a helyes válaszokat!

- ✔ A kód modularitása növekszik, mivel az egyes funkciók és felelősségi körök elkülönülnek egymástól
- ✔ Könnyebbé válik a rendszer karbantartása és bővítése, mivel az egyes részek izoláltak
- ✔ Csökkenti a rendszer komplexitását, mivel minden modul saját felelősséggel bír
- ✘ Megnöveli a rendszer futási idejét, mivel a különböző modulok közötti kommunikáció gyakran lassú

Kérdés 2: Mi a legfontosabb célja a Separation of Concerns elvének?

- ✔ A különböző funkcionalitások és felelősségi körök egyértelmű elkülönítése
- ✘ A rendszer minden aspektusának egyetlen helyen történő kezelésére összpontosít
- ✔ Csökkenti az összetettséget, és elősegíti a kód tisztaságát
- ✘ A programok gyorsabb végrehajtását célozza meg

Kérdés 1: Mi a Dependency Injection (DI) fő célja a szoftverfejlesztésben? Jelölje meg a helyes válaszokat!

- ✔ Az osztályok közötti laza összekapcsolás elősegítése, hogy a különböző komponensek könnyebben cserélhetőek legyenek
- ✔ Az alkalmazás kódjának optimalizálása a futási sebesség szempontjából
- ✘ Az összes függőség kézi kezelése, ami a kód szorosabb összekapcsolását eredményezi
- ✔ Az osztályok tesztelhetőségének növelése azáltal, hogy könnyebbé válik a mock vagy stub objektumok alkalmazása

Kérdés 2: Mi jellemzi a Dependency Injection-t?

- ✔ A függőségeket nem a komponens magában hozza létre, hanem kívülről kapja meg
- ✔ A függőségeket minden egyes osztály kezeli
- ✘ Az alkalmazás komponensei közvetlenül kommunikálnak egymással a függőségek kezelésére
- ✔ A komponens felelőssége, hogy meghatározza, hogyan lesznek a függőségek injektálva

Kérdés 1: Mi a Composite tervezési minta fő célja? Jelölje meg a helyes válaszokat!

- ✔ Az objektumokat hierarchikus struktúrába szervezni, hogy azok egyesíthetők legyenek
- ✔ Lehetővé tenni, hogy az egyedi objektumok és a komponensekből álló összetett objektumok ugyanúgy kezelhetők legyenek
- ❌ A kódot minden egyes objektumra külön-külön kell írni
- ✔ Az objektumok közötti laza összekapcsolás elősegítése, hogy az összetett objektumok cserélhetőek legyenek

Kérdés 3: Mi a fő előnye a Composite mintának?

- ✔ Az egyes elemek (levelek) és összetett elemek (kompozitok) egyszerű és egységes kezelése
- ✔ Segít a komplex struktúrák egyszerűbb kezelésében, mivel az összetett objektumok átláthatóan építhetők fel
- ❌ Az összetett objektumok és levelek közötti különbségek bonyolítják a kódot
- ✔ A rendszer könnyen bővíthető új típusú objektumokkal, anélkül hogy módosítani kellene a meglévő kódot

Kérdés 1: Mi a Memento tervezési minta fő célja? Jelölje meg a helyes válaszokat!

- ✔ Az objektumok állapotának mentése és visszaállítása anélkül, hogy a belső működést felfednénk
- ✔ Az objektumok állapotának megváltoztatása külső beavatkozás nélkül
- ❌ A rendszer összes objektumának állapotát tárolni, hogy bármikor visszaállítható legyen
- ✔ A komplex objektumok állapotainak kezelésének egyszerűsítése

Kérdés 4: Mi a Caretaker szerepe a Memento mintában?

- ✔ Az állapotokat tárolja, de nem változtat rajtuk
- ❌ Ő végzi el az állapotok visszaállítását az eredeti objektumban
- ✔ Az állapotok mentését és visszaállítását más objektumok végzik, a Caretaker csak tárolja őket
- ❌ Az ő feladata az eredeti objektum viselkedésének kezelését

Kérdés 3: Mi az **Adapter** minta előnye?

- ✔ Lehetővé teszi, hogy régi vagy külső osztályokat egy újabb rendszerben használjunk anélkül, hogy azokat módosítanunk kellene
- ✔ Csökkenti az osztályok közötti kommunikációs költségeket
- ✔ A rendszer bővíthetőségét és karbantarthatóságát javítja
- ❌ Tökéletes megoldást nyújt a rendszer összes interfészének kezelésére

Kérdés 4: Mi az **Document-View** minta előnye?

- ✔ Segíti a felhasználói felület és az adatmodellek elválasztását, így könnyebben karbantartható
- ✔ Lehetővé teszi, hogy az adatokat és nézeteket függetlenül kezeljük
- ✔ Csökkenti a program komplexitását
- ❌ Nem bővíthető könnyen különböző dokumentumformátumokkal

MVC (Model-View-Controller) minta

Kérdés 1: Mi az **MVC** minta alapvető célja?

- ✔ A felhasználói felület, az adat és a vezérlési logika elválasztása
- ✔ A felhasználói felület és az adat szoros összekapcsolása
- ✔ Az adatok és a vezérlési logika közötti közvetlen kapcsolat fenntartása
- ✔ Az adatkezelés kizárólag a nézetben történik

MVVM (Model-View-ViewModel) minta

Kérdés 1: Mi az MVVM minta alapvető célja?

- ✔ A felhasználói felület és az üzleti logika elválasztása a köztes **ViewModel** réteggel
- ✔ A felhasználói felület közvetlenül kommunikál az adatmodellel
- ✔ Az adatot kizárólag a **View** tartalmazza
- ✔ A nézet közvetlenül kezeli az üzleti logikát